

Тема урока: наследование

1. Переопределение методов
2. Расширение методов
3. Функция `super()`
4. Использование методов наследников в базовом классе

Аннотация. Урок посвящен наследованию в Python.

Переопределение методов

В прошлом уроке мы познакомились с концепцией наследования, а также дали определения родительским и дочерним классам. Выяснили, что дочерние классы, наследуя методы родительских классов, могут дополнять их собственными методами, а также переопределять уже имеющиеся.

Рассмотрим класс `Animal`, описывающий животное, и его подкласс `Cat`, описывающий кошку:

```
class Animal:
    def __init__(self, name, age):
        self.name = name           # имя животного
        self.age = age             # возраст животного

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def sleep(self):
        return f'{self.name} очень крепко спит zZ'

    def sound(self):
        return 'Мяя!'
```

Класс `Cat`, являясь наследником класса `Animal`, наследует метод `__init__()` родительского класса, переопределяет метод `sleep()` родительского класса и дополнительно определяет метод `sound()`.



Переопределять в дочернем классе можно как обычные методы, так и магические.

Переопределяя методы, мы можем менять их сигнатуру, например, добавляя дополнительные параметры или наоборот уменьшая их количество. Важно понимать, что переопределяя метод в дочернем классе, мы полностью его заменяем, а не получаем два разных рабочих метода. Таким образом, для экземпляров дочернего класса вызывается метод, определенный именно в дочернем классе, а метод, определенный в родительском классе, становится недоступен.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def sleep(self, times):
        return f'{self.name} спит {times} часа zZ'

cat = Cat('Кемаль', 1)

print(cat.sleep(2))
```

ВЫВОДИТ:

```
Кемаль спит 2 часа zZ
```

В то время как приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def sleep(self, times):
        return f'{self.name} спит {times} часа zZ'

cat = Cat('Кемаль', 1)

print(cat.sleep())
```

приводит к возбуждению исключения, так как метод `sleep()` для экземпляров класса `Cat` должен принимать один обязательный аргумент:

TypeError: Cat.sleep() missing 1 required positional argument: 'times'

Расширение методов

Часто в дочернем классе мы хотим не полностью заменить метод родительского класса, а лишь расширить его. В таких случаях решением является вызов метода базового класса в теле соответствующего метода наследника и добавление дополнительного кода.

Вернемся к классу `Animal` и его наследнику классу `Cat`. Если мы определим для экземпляров класса `Animal` атрибуты `name`, `age` и `eyecolor`, а для экземпляров класса `Cat` захотим добавить дополнительный атрибут `breed`, нам будет нужно соответствующим образом переопределить инициализатор в дочернем классе.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age, eyecolor):
        self.name = name           # имя животного
        self.age = age             # возраст животного
        self.eyecolor = eyecolor  # цвет глаз животного

class Cat(Animal):
    def __init__(self, name, age, eyecolor, breed):
        self.name = name
        self.age = age
        self.eyecolor = eyecolor
        self.breed = breed        # порода кошки

animal = Animal('Роджер', 2, 'черный')
cat = Cat('Кемаль', 1, 'синий', 'манчкин')

print(animal.name, animal.age, animal.eyecolor)
print(cat.name, cat.age, cat.eyecolor, cat.breed)
```

ВЫВОДИТ:

```
Роджер 2 черный
Кемаль 1 синий манчкин
```

Так как часть кода в инициализаторе дочернего класса является копией кода в инициализаторе родительского класса, правильным будет вызов инициализатора базового класса в инициализаторе наследника, а не дублирование кода.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age, eyecolor):
        self.name = name
        self.age = age
        self.eyecolor = eyecolor

class Cat(Animal):
    def __init__(self, name, age, eyecolor, breed):
        Animal.__init__(self, name, age, eyecolor)
        self.breed = breed

animal = Animal('Роджер', 2, 'черный')
cat = Cat('Кемаль', 1, 'синий', 'манчкин')

print(animal.name, animal.age, animal.eyecolor)
print(cat.name, cat.age, cat.eyecolor, cat.breed)
```

ВЫВОДИТ:

```
Роджер 2 черный
Кемаль 1 синий манчкин
```

Здесь мы напрямую обращаемся к родительскому классу `Animal` и его методу `__init__()`, передавая экземпляр класса `Cat` и значения атрибутов, которые следует установить.

Также расширение метода может быть полезно в том случае, когда экземплярам дочернего класса мы хотим не добавить дополнительный атрибут, а установить значения по умолчанию уже имеющимся, тем самым уменьшить количество передаваемых аргументов при создании экземпляров класса наследника. Наиболее удачным примером в данном случае служат прямоугольник и квадрат. Например, мы имеем класс `Rectangle`, описывающий прямоугольник и его дочерний класс `Square`, описывающий квадрат:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length          # длина прямоугольника
        self.width = width            # ширина прямоугольника

class Square(Rectangle):
    pass
```

Квадрат является прямоугольником, у которого длина и ширина совпадают, поэтому было бы удобно при создании экземпляров класса `Square` передавать лишь одну сторону, а вторую считать равной переданной. Для этого мы можем определить в классе `Square` метод `__init__()`, принимающий один аргумент — сторону квадрата, а в теле этого метода вызвать метод `__init__()` класса `Rectangle`, передав ему эту сторону в качестве длины и ширины.

Приведенный ниже код:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

class Square(Rectangle):
    def __init__(self, side):
        Rectangle.__init__(self, side, side)

rectangle = Rectangle(3, 4)
square = Square(2)

print(rectangle.length, rectangle.width)
print(square.length, square.width)
```

ВЫВОДИТ:

```
3 4
2 2
```

Расширение метода на примере инициализатора наиболее наглядно с практической точки зрения, однако, конечно, расширить можно абсолютно любой метод.

Функция `super()`

Расширение методов в том виде, в котором было показано ранее, хоть и рабочее, однако имеет небольшой недостаток, который заключается в том, что мы напрямую обращаемся к родительскому классу и конкретному методу:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Cat(Animal):
    def __init__(self, name, age, breed):
        Animal.__init__(self, name, age)           # явно обращаемся к методу
__init__() класса Animal
        self.breed = breed
```

Такой подход может привести к ошибкам, например, если родительский класс изменит свое имя или если у дочернего класса родительским станет другой класс. Во избежание подобных проблем в Python используется встроенная функция `super()`, которая позволяет нам неявно обращаться к родительскому классу. Наиболее распространенным вариантом ее применения является именно вызов метода родительского класса из метода дочернего класса.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Cat(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age) # неявно обращаемся к методу
        # __init__() родительского класса
        self.breed = breed

cat = Cat('Кемаль', 1, 'манчкин')

print(cat.name, cat.age, cat.breed)
```

демонстрирует использование функции `super()` и выводит:

```
Кемаль 1 манчкин
```

В данном случае выражение `super().__init__(name, age)` равнозначно `Animal.__init__(self, name, age)` и звучит так: вызови метод `__init__()` у моего родительского класса и передай ему в качестве аргументов текущий экземпляр `self`, а также значения `name` и `age`. То есть объект, возвращаемый функцией `super()`, связывает текущий класс `Cat`, экземпляр класса `Cat`, доступный по имени `self`, и родительский класс `Animal`. Также следует обратить внимание, что при вызове родительского метода с помощью функции `super()`, экземпляр класса в качестве первого аргумента передавать не нужно.



Объект, возвращаемый функцией `super()` часто называют прокси-объектом или объектом-посредником. Его целью является осуществление доступа к родительскому классу текущего класса.

По умолчанию функция `super()` сама определяет текущий класс, его родительский класс и текущий экземпляр текущего класса, и в примере выше мы наблюдали данное поведение. Однако текущий класс и текущий экземпляр этого класса мы можем указывать явно.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Cat(Animal):
    def __init__(self, name, age, breed):
        super(Cat, self).__init__(name, age)
        self.breed = breed

cat = Cat('Кемаль', 1, 'манчкин')

print(cat.name, cat.age, cat.breed)
```

ВЫВОДИТ:

Кемаль 1 манчкин

В примере выше в качестве первого аргумента мы передаем класс `Cat`, тем самым указывая, что к методу `__init__()` нужно обратиться именно у родительского класса `Cat`. В качестве второго аргумента мы передаем текущий экземпляр класса `Cat` — `self`, тем самым указывая, какой именно объект нужно передать в функцию `__init__()` в качестве первого аргумента.



Функция `super()` так названа в честь названия родительского класса `superclass`.

Отличительной особенностью функции `super()` является то, что она предоставляет доступ, скорее, не к конкретному классу, а ко всей иерархии классов. И если в нашем наследовании участвует три или более классов, то функция `super()` выполняет поиск необходимого метода в каждом из этих классов.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name):
        print('Вызов метода __init__() класса Animal')
        self.name = name

class Cat(Animal):
    pass

class Kitten(Cat):
    def __init__(self, name, breed):
        print('Вызов метода __init__() класса Kitten')
        super().__init__(name)
        self.breed = breed

cat = Kitten('Кемаль', 'манчкин')

print(cat.name, cat.breed)
```

ВЫВОДИТ:

```
Вызов метода __init__() класса Kitten  
Вызов метода __init__() класса Animal  
Кемаль манчкин
```

В примере выше функция `super()` сперва проверяет наличие метода `__init__()` в классе `Cat`, а затем в классе `Animal`.

Использование методов наследников в базовом классе

Рассмотрим класс `Animal` и двух его наследников `Cat` и `Dog`:

```
class Animal:  
    def __init__(self, name):  
        self.name = name  
  
class Cat(Animal):  
    def sound(self):  
        return 'мяу'  
  
class Dog(Animal):  
    def sound(self):  
        return 'гав'
```

Экземпляры дочерних классов имеют атрибут `name` с именем животного и определяют метод `sound()`, возвращающий звук, издаваемый соответствующим животным. Предположим, мы хотим определить метод `info()`, который объединяет данную информацию, а именно имя и звук, и возвращает их в виде одной строки. Первым решением было бы определение данного метода как в классе `Cat`, так и в классе `Dog`.

Приведенный ниже код:


```
class Animal:
    def __init__(self, name):
        self.name = name

class Cat(Animal):
    def sound(self):
        return 'мяу'

    def info(self):
        return f'Имя: {self.name}, звук: {self.sound()}'

class Dog(Animal):
    def sound(self):
        return 'гав'

    def info(self):
        return f'Имя: {self.name}, звук: {self.sound()}'

cat = Cat('Кемаль')
dog = Dog('Роджер')

print(cat.info())
print(dog.info())
```

ВЫВОДИТ:

```
Имя: Кемаль, звук: мяу
Имя: Роджер, звук: гав
```

Однако если при наследовании дочерние классы определяют некоторые общие методы, их удобнее и правильнее выносить в родительский класс.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def info(self):
        return f'Имя: {self.name}. Звук: {self.sound()}'
```



